

对比学习方向的论文对标

第 3 阶段工作汇报 · 2026-09-12 · CMU-MOSI / CMU-MOSEI

一、这一阶段的任务与交付

任务是复现 8 篇含对比学习的论文做对标，具体哪 8 篇由我们自己定。本报告给出名单、分档、已完成的复现结果，以及一条比"复现了几篇"更重要的发现。

选取标准三条：(a) 方法里确有对比学习项（不是任意的辅助损失）；(b) 在 MOSI / MOSEI 上报了回归指标，否则与我们的协议无法并列；(c) 覆盖 2021–2024，并优先有公开代码的——**能跑通的才叫复现**。

档	论文	会议 / 年份	对比学习是什么	代码
A	MMIM	EMNLP 2021	分层互信息最大化，两个 InfoNCE 形式的下界项	有
A	ConFEDE	ACL 2023	特征分解 + 样本间对比	有
A	HSCL	MMM 2024	分层有监督对比（模态内 / 模态间 / 样本间）	有
C	UniMSE	EMNLP 2022	模态间 + 样本间对比，统一 MSA 与 ERC 标签空间	有但不完整
C	HyCon	IEEE TAFRC 2022	模态内 / 模态间对比 + 半对比	无
C	ConKI	ACL Findings 2023	知识注入 + 多层次对比	无
C	CMSBERT-CLR	ICASSP 2022	上下文驱动模态偏移 + 对比对齐	无
C	MMCL	arXiv 2022.10	单模态对比编码 + 跨模态对比预测	无

A 在我们自己 sha256 校验过的特征上真的跑起来了，是唯一能与我们自己的结果同表并列的档。
B 原本是"代码公开、值得移植"这一档，现在是空的——它只有 UniMSE 一篇，已于 2026-09-11 降入 C 档（第四节）。
C 无法在统一特征上复现，只能引用论文自己报的数字，一律标注为"待检验的主张"。

档	篇数	论文
A 在我们校验过的特征上跑通	3	MMIM、ConFEDE、HSCL
B	0	——
C 只引用论文报告值	5	UniMSE、HyCon、ConKI、CMSBERT-CLR、MMCL

这个计数本身是结论的一部分。八篇里 四篇从未公开代码，第五篇（UniMSE）公开了但一个 batch 都跑不起来——而它恰好是全表 MOSI 报告值最好的那一篇（MAE 0.691）。报告值的高低，与那个数字可被检验的程度之间没有关系。这句话现在是数出来的。

为什么必须分档。把我们测的数字和论文自己报的数字放进一张无标注的表，是这个方向最容易犯、也最难被外人发现的错误。理由不是谨慎，是实测：同一个 MMIM，我们的实现比参照实现

好约 1 个标准差，原因排除了 8 个候选后至今没定位。跨实现的 0.01 量级差异说明不了方法优劣。

二、8 篇论文自己报告的数字

全部从 PDF 抄录、注明表号，不凭记忆填。

论文	CMU-MOSI				CMU-MOSEI				出处
	MAE↓	Corr↑	Acc-7↑	Acc-2 (has0/non0)	MAE↓	Corr↑	Acc-7↑	Acc-2	
MMIM	0.700	0.800	46.65	84.14/86.06	0.526	0.772	54.24	82.24/85.97	Table 2
ConFEDE	0.742	0.784	42.27	84.17/85.52	0.522	0.780	54.86	81.65/85.82	Table 2
HSCL	未取得 — MMM 2024 在 Springer 付费墙后，README 未列								—
UniMSE	0.691	0.809	48.68	85.85/86.90	0.523	0.773	54.39	85.86/87.50	Table 2
HyCon	0.713	0.790	46.6	—/85.2	0.601	0.776	52.8	—/85.4	Table 1,2
ConKI	0.681	0.816	48.43	84.37/86.13	0.529	0.782	54.25	82.73/86.25	Table 2
CMSBERT-CLR	0.708	0.802	—	84.82/86.56	0.523	0.776	—	84.39/86.69	TABLE I
MMCL	0.705	0.797	46.5	84.0/86.3	0.537	0.765	53.6	84.8/85.9	Table 1

只有 HyCon 与 MMCL 明确写了"五次运行取均值"，其余多数未说明 seed 数。在 MOSI 上报单次运行的数字，等于报一个 ± 0.039 MAE 的量（我们实测的 LF-LSTM seed 间标准差），比这张表里 0.681 到 0.742 的全部跨度还宽。

三、已完成的复现

ConFEDE (ACL 2023) — 已复现

原作者代码逐字节未改，读我们 sha256 校验过的 MOSI 特征，用它自己的 5 个 seed。

指标	我们 (n=5)	论文 Table 2	差
MAE ↓	0.7379 ± 0.0050	0.742	-0.0041
Corr ↑	0.7805 ± 0.0007	0.784	-0.0035
Acc-7 ↑	44.90 ± 0.46	42.27	+2.63
Acc-2 has0 ↑	83.36 ± 0.16	84.17	-0.81
Acc-2 non0 ↑	84.91 ± 0.15	85.52	-0.61

MAE 与 Corr 都落在论文值 0.004 以内，Acc-7 高 2.6 个点。判定：已复现。

但那个 seed 标准差不能按字面读。Corr 的 ± 0.0007 比我们任何模型都小一个量级（LF-LSTM 在 MOSI 上是 ± 0.039 MAE）。原因在它代码里：main.py 没有 seed 循环——三个编码器只预训练一次，五个融合 seed 加载的是同一套预训练权重。那个方差只覆盖融合阶段的随机性，不是整条流水线的。

HSCL (MMM 2024) —— 已跑通，并发现测试集泄漏

它读的是 CMU-MultimodalSDK 那一系的数据布局，键名、id 格式、标签形状、补零方向都与我们的不同。方向固定为改我们的数据去适配它的代码，不改它的代码——两边读同一份校验过的特征，指标差异才不能归因于输入。

跑通之后读它的 solver.py，发现上报的那一轮必须同时"验证集变好"且"测试集变好"：

```
if val_loss < best_valid:
    best_valid = val_loss
    if test_loss < best_mae: ← 测试集损失参与决定选哪一轮
        best_results = results ← 上报的就是这一轮的测试集预测
```

这是测试集泄漏进模型选择，违反我们的约定"模型选择只看验证集"。在默认 seed 那次运行里能直接看到它发生：第 20 轮验证损失 0.7031 是全程最低、干净协议下就该选它，但它的测试损失 0.7282 比第 14 轮的 0.7197 差，于是上报第 14 轮。

这不是指控作者作弊，这类写法在本领域开源代码里很常见，往往从模板继承而来。注意它是在有代码的 A 档里靠读代码发现的；C 档那四篇没有代码，连查都查不了。

因此做了两臂各 5 seed (seeds 42–46)，唯一差别是那一行：as-released（发布版）与 clean（只按验证集选）。

指标	as-released (发布版)	clean (只看验证集)	差	p 双侧	MDE	判定
MAE ↓	0.7371 ± 0.0183	0.7315 ± 0.0205	+0.0057	0.656	0.0338	小于 MDE, 不可读
Corr ↑	0.7811 ± 0.0104	0.7895 ± 0.0089	-0.0084	0.208	0.0169	小于 MDE, 不可读
Acc-7 ↑	45.74 ± 1.48	45.25 ± 1.13	+0.50	0.569	2.29	小于 MDE, 不可读
Acc-5 ↑	51.60 ± 1.49	51.46 ± 1.64	+0.15	0.887	2.72	小于 MDE, 不可读
Acc-2 has0 ↑	81.92 ± 0.90	82.22 ± 0.88	-0.29	0.618	1.55	小于 MDE, 不可读
Acc-2 non0 ↑	83.78 ± 1.08	83.84 ± 1.15	-0.06	0.934	1.94	小于 MDE, 不可读

六个指标的观测效应全部小于各自的最小可检测效应，方向不可读（四个偏 clean、两个偏发布版，p 全在 0.2 以上）。用非配对 Welch——配对的前提（两臂轨迹相同）实测不成立，见下节。

这里我抢跑了两次，如实记下。

第一次：我从机制推断"泄漏必然抬高论文数字"。方向是反的。clean 上报的是验证损失最低的那一轮；as-released 要求验证改善且测试改善，接受集是 clean 的子集，所以它上报的轮次不晚于 clean——那个合取条件让它提早冻结在训练不足的一轮上。

第二次：跑到两个 seed 时看到 clean 明显更好（MAE 0.7020 对 0.7127），我据此写了 "clean 更好"。没撑到第五个 seed。

两次都是同一个毛病：在预先定好的样本量跑完之前读方向。判据早就写好，是我自己没守。

为什么测不出来：这个模型固定 seed 也不可复现

查两臂差异时撞见的，比两臂的结论本身更要紧。两臂的 per-epoch 轨迹从第 1 轮就不同，而第 1 轮那行打印在任何 save 分支之前——不可能是补丁造成的。于是直接测：同一 seed、同一份发布版代码、连跑两次。

发布版, seed 42	第 1 轮 valid	第 1 轮 test	上报 MAE
run 1	1.1825	1.1080	0.7019
run 2	1.1794	1.1070	0.7098
20 轮中逐位相同	0 轮		差 0.0079

原因：它的 `set_seed()` 设了 `cuda.deterministic = True`，但那只管 cuDNN 卷积——管不到 BERT 与 TransformerEncoder 反向里的原子加，那要靠 `use_deterministic_algorithms(True)`，它从未设置。

这 0.0079 是关键的一个数。它是同一份代码、同一个 seed、什么都没改，仅重跑一次就产生的差异。而它比这份对标表里好几个"改进"还大——MMCL 相对 MMIM 是 0.005，CMSBERT-CLR 相对 Self-MM 是 0.005。

对照我们自己的做法：`check_repro.py` 是 12 道闸门之一，专门问“同一台机器 连跑两次是否逐比特一致”；LF-LSTM 默认路径的预测哈希 `172967c7dced83b2` 贯穿多次重构未变。**这不是我们比谁讲究——没有这道闸门，“改动 A 带来了 0.005 的提升”这句话根本无法成立**，而这整个阶段读到的就是这句话被反复说出来。

UniMSE：兼容性全解决了，卡点不在那里

它原定 B 档（代码公开、值得移植）。兼容层做完后——四个搬了家的 `transformers` 模块 做别名（25 个下游符号逐一核对存在后才做）、`plot_confusion_matrix` 注入调用即抛异常的桩、`ReduceLROnPlateau(verbose=)` 用与 `MMSA` 那处同一种写法的补丁——它按我们的划分加载了数据（1284/229/686）、载入 `T5-base`、建好 `adapter` 与 `T5` 参数分组、建好 `optimizer` 与 `scheduler`，然后死在第一个 batch：

```
data_loader.py:260 [task_prefix + sequence for sequence in inputs_seq]
TypeError: can only concatenate str (not "list") to str
```

`create_dataset.py` 把文本存成单词列表，而 `collate_fn` 拿它去和字符串前缀相加。旁边还有第二处不一致：`score = str(sample[1])` 会得到 `"[-0.5]"` 这种 `T5` 目标串，而被注释掉的那行写的是 `str(sample[1][0][0])`。这条路径不是作者跑的那条——他们跑的是 `data_processor.py`，读仓库里没有、只在网盘链接里的预处理文件。

补那两行的念头，是按证据否掉的。`solver.py` 里 `pre_gen` 与被注释的测试分支把生成串按逗号拆成 `por / score / meld / iemocap` 四段——`T5` 的目标是跨四个数据集的逗号分隔复合标签，正是论文那个“统一标签空间”，由它的 `SimCSE` 步骤生成。补那两行等于替作者发明论文的核心贡献，产出的数字不是 UniMSE 的数字。README 自己写着 “we will open the source codes in the future”——这是明确的部分发布，不是我们配错了。

下载它自己的特征是活路，但回答的是另一个问题。它的预处理 `pickle` 把统一标签烘进去了，我们的特征没法替换进去，A 档定义永远达不到。用它的特征跑，问的是“它的流水线能否复现它自己报的数字”，而不是“它的方法在与别人 相同特征上如何”。两个都值得知道，但都不能放进与我们数字并列的表。

四、核心发现：这个方向“改进”的报告口径宽于测量精度

六条独立证据，都来自这一阶段：

1. 同一个已发表的 MMIM，在 MOSI 上被报成两个数。MMIM 自己（unaligned）报 0.700，MMCL 的基线表（token-aligned）报 0.738。两边都没错——设置不同。但差 0.038，是我们 MOSI 分辨极限（最小可检测效应 0.013）的三倍。
2. HyCon 自己的消融声称对比学习值 0.056 MAE（去掉全部对比损失，MOSI MAE 从 0.713 涨到 0.769）。这是我们 MOSI 上 MDE 的四倍多——如果它在我们的协议下成立，我们不可能测不出来。而我们在 MMIM 上做的受控 on/off 对照（同一模型、已证明与参照实现逐比特一致、n=20）测到的是 MOSI 上 0.0046 不显著、MOSEI 上显著有害 ($|d|=1.39$)。

3. HSCL 的模型选择读测试集（上一节）。泄漏是真的，但它对上报指标 是抬高还是压低，在 $n=5$ 下测不出来——要分辨那 0.0057 的 MAE 差，每臂需要 $n \approx 143$ 、共约 12 小时。那个数字本身就是结论：这条规则的效应小于 该模型自身运行噪声在任何合理预算下能分辨的量。
4. HSCL 固定 seed 也不可复现，重跑一次上报 MAE 就差 0.0079（上一节）——比表里好几个"改进"还大。它自己单 seed 之间的 MAE 极差更是有 0.039，比整张对标表 0.681 到 0.742 的全部跨度还宽。
5. ConFEDE 的 5 seed 方差不覆盖预训练阶段（上一节）—— `main.py` 没有 seed 循环，五个融合 seed 共用同一套预训练编码器。
6. 八篇里只有三篇能在统一特征上跑通：四篇从未公开代码，第五篇 公开了但跑不到一个 batch——而那一篇的 MOSI 报告值是全表最好的。

合起来是一个连贯的结论。这既解释了为什么第 2 阶段 在损失函数上反复测不出东西——不是方法不行，是那个效应量本来就在噪声以下——也说明对标真正该回答的问题不是"谁的数字更高"，而是"这些论文报的对比学习收益，在一个受控、多 seed、可复现的协议下还剩多少"。

五、下一步与排期

八篇已全部处理完毕，分档定稿为 3 / 0 / 5。A 档的完整流水线是五步：跑通 → 与论文值比 → 移植进我们的注册表 → 数值等价测试（权重复制 + 前向比对）→ 多 seed。ConFEDE 与 HSCL 停在第 2 步，MMIM 走完了全部五步。

ConFEDE 的移植成本已勘定：一整天量级。三处与我们的模型契约不兼容——它的 `forward` 一次吃两批（每个锚点配 6 个伴随样本）、检索集是预训练编码器的中间产物、四阶段流水线。约 600 行要重写而非转抄，因此必须做数值等价测试，而四个阶段都要对上是最难的一环。

覆盖面已做完，移植仍建议推后。理由被这一轮加强了：ConFEDE 与 HSCL 上全部四条有价值的发现——共用预训练、模型选择读测试集、固定 seed 不可复现、效应小于 自身噪声——**没有一条是移植换来的**，都来自读它们的代码、以及在它们自己的 代码上做受控对照。移植买的是绝对数字可并列，只有当我们写"统一协议下的横向表"时才成为必需。

所有数字可从仓库重现：`bash scripts/setup_paper_repros.sh` 配置，`bash scripts/run_hscL_arms.sh` 跑 HSCL 两臂。名单与分档见 `docs/benchmark_contrastive.md`，排查记录见 `docs/investigations.md`（锚点 `#hscL-test-leak`、`#mmsa-padding-not-zero`、`#confede-missing-ckpt-dir`、`#paper-repro-compat`）。